Student IDs : 60216681; 60915652; 60875477; 61076721

Student Names : WANG Shen; LIU Wanyang; WANG Yuwei; YU Xiyan

Student Emails : swang2428-c@my.cityu.edu.hk; wanyanliu4-c@my.cityu.edu.hk; ywang8484-c@my.cityu.edu.hk; xiyanyu3-c@my.cityu.edu.hk;

Title:

One Core to Many: Parallel Data-Computing Solutions to the 2023 NYC Yellow-Taxi Trips

Abstract:

One process cleans and aggregates the twelve yellow-taxi files of 2023 in 52.135 seconds. Those files hold 38,310,226 rows before cleaning and 34,893,980 after seven exclusive rules, so 8.92 percent are removed. Of the dated pickups, 89.8 percent start in Manhattan and 9.79 percent start in Queens. Weekdays hold 72.9 percent, and the busiest cell is Thursday at 18:00, with 405,828 trips. On zone location, time of day, a weekend indicator, and log distance, k-means reaches its silhouette maximum at k = 2 (0.550). That cut is the JFK tail against the rest. We report k = 4 (silhouette 0.312): weekday morning, weekday afternoon and evening, weekend, and a JFK group whose mean trip is 16.1 miles and 63.69 dollars. Zone 132, JFK Airport, accounts for 98.3 percent of that group. On this four-core host the process-pool ETL falls from 52.135 seconds at one worker to 13.658 seconds at four (speedup 3.82) and rises to 14.266 seconds at eight. Four threads take 27.423 seconds. A chunk that contains all twelve files takes 49.668 seconds. The k-means kernel falls from 6.428 seconds to 1.611 seconds (speedup 3.99). FP-Growth does not speed up: at four workers the speedup is 0.89, because the prefix tree is built before the pool starts.

Introduction

One process cleans and aggregates the twelve yellow-taxi files of 2023 in 52.1 seconds. That is the workload we parallelize. The same files are also a table of trips. We ask whether k-means separates those trips by place and by time of day.

The files are the TLC yellow-trip release for January through December 2023 [1]. They contain 38,310,226 rows before cleaning and 34,893,980 rows after it. We use the TLC taxi-zone lookup to test zone ids and to name boroughs, and the TLC zone shapefile to place a centroid in each zone. The cleaning code, the parallel kernels, the figures, and the measured runs are open source at Github repository.

Eight workers on four cores is oversubscription. We report that point, and we do not treat the machine as an eight-core server. FP-Growth is timed only as a split that does not speed up. We do not use its itemsets as a

result.

Motivation

In this report, we try to give a parallel solution to a data set that is large enough for the schedule to show in the wall time. One worker cleans and aggregates these twelve files in a median of 52.135 seconds. That is long enough to time, and short enough that the repeats fit on one host.

The cleaning rules are implemented by Python codes [3]. The GIL lets one thread run that bytecode at a time. Numeric kernels inside NumPy can release it [4], so a thread pool speeds up only the part of the ETL that has already released the lock. A process pool gives each worker its own interpreter. The measurements below are there to show which of those two pools, and which chunk size, uses the four cores.

A step that stays in one process does not get shorter when the pool grows. Amdahl's observation is that statement applied to a fixed problem [5]. It is why an ETL chunk that contains every file, and an FP-Growth tree built in the parent, leave the extra workers idle.

Objective

The report aims to answer three questions.

1. How is yellow-taxi demand in 2023 arranged in time and across boroughs?

2. On a feature set that includes zone location and time of day, what groups does k-means produce at k = 4, and how does that cut compare with the silhouette maximum?

3. On this 4-core host, how does ETL wall time change with the number of processes, with a thread pool, and with the chunk size? How does the k-means kernel scale from 1 worker to 4?

Speedup and efficiency are S(p) = T1 / Tp and E(p) = S(p) / p, where Tp is the median wall time with p workers after one discarded warmup. We use those two summaries. We do not fit a limiting speedup for a machine we did not run.

Data Sources

The trip files were downloaded on 9 November 2026 from the TLC CloudFront bucket. Each file is one calendar month of 2023. Row counts are Parquet metadata. The shapefile supplies the zone centroids used as clustering features and as the cluster map. The TLC records do not contain passenger or driver identifiers.

Source	Role	Rows
TLC yellow trips, 2023-01 to 2023-12	Primary table	38,310,226

TLC taxi zone lookup	Borough and valid zone ids	265
TLC taxi zone shapefile	Zone centroids	263 polygons

Table 1. Sources for the 2023 study. Trip rows are the raw Parquet metadata total.

The pipeline keeps pickup time, dropoff time, passenger count, trip distance in miles, rate code, pickup zone, dropoff zone, payment type, fare, tip, and total amount. Derived fields are duration, hour, a weekend indicator, and the zone centroid. Distance, duration, and fare for the histograms are the cleaned rows of these twelve files. The cluster sample is a draw of 25,000 cleaned rows from each file.

Methodology

Cleaning rules

Rules are applied in order. A row is charged to the first rule that rejects it, so the seven counts sum to the number of removed rows. Duration must lie between 1 minute and 3 hours. Distance must be positive. Both zone ids must appear in the TLC lookup and must not be the Unknown borough. Fare must lie between 2.50 and 500 dollars. Speed must be positive and strictly below 70 mph. Duplicate keys, on pickup, dropoff, both zones, distance, and fare, are removed inside each 1,000,000-row batch, not across batches. Invalid codes cover a passenger count outside 1–6 when the field is present, a rate code outside 1–6 when present, and a payment type outside 1–4. A zero in one of those fields is the file's marker for a value that was not recorded.

ETL

At file scale the split is data parallel. Each worker receives a different subset of the rows and runs the same code. The map reads one monthly file, applies the cleaning rules, and emits counts by pickup zone, hour, and day of week. The reduce adds those counts. Workers return arrays indexed by zone, not the trips, so the bytes that cross the process boundary stay small. Dean and Ghemawat's MapReduce is the same shape: map, shuffle, and reduce [2].

Figure 1 displays the ETL process. Figure 2 names the timed workloads. The ETL and the k-means kernel are the two we use. FP-Growth is the third curve on the speedup plot, and it is not a result.

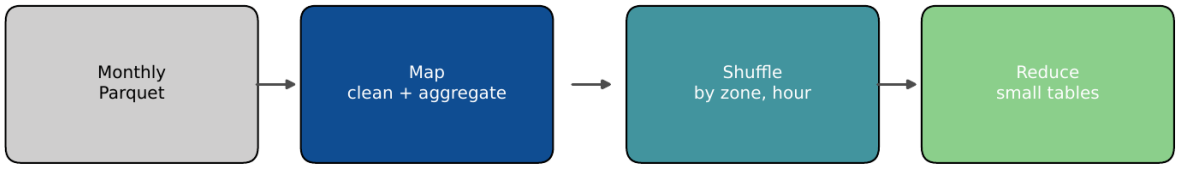


Figure 1. ETL as map, shuffle, and reduce. Each worker returns zone aggregates, not trips.

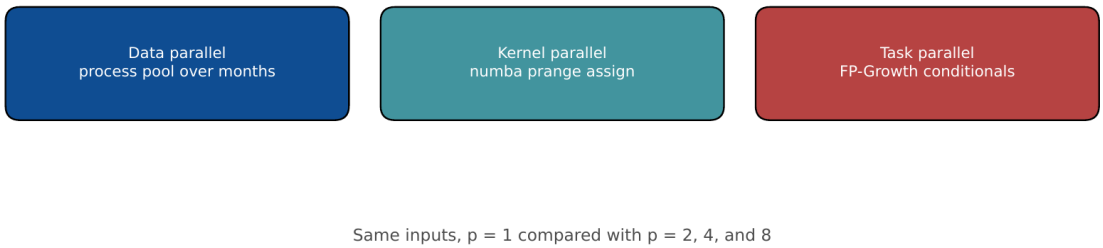


Figure 2. The timed workloads. They do not share a scheduler. Each has its own p. The reported analysis uses the ETL and k-means.

The Parquet files have a single row group, so a month is not split by row group. Chunk size is the number of files handed to a worker in one batch. With twelve files, a chunk size of 12 assigns the whole year to one worker. The comparison is three settings of the same twelve files. The process pool varies p at chunk size 1. The thread pool is run at p = 4. The chunk-size runs use the process pool at p = 4 with chunk size 4 and chunk size 12. A pure-Python loop of 400,000 tuples, with no NumPy, is timed separately so the GIL claim is not only a property of pandas.

k-means

Lloyd's algorithm alternates assignment and centroid update [6]. Initial centers use k-means++ on a 20,000-row draw [7]. The assignment is a parallel for over blocks of rows: each point reads every center and writes one label [8]. The published labels reduce centroids in row order, so the labels do not depend on the thread count. The timed kernel reduces with a per-thread accumulator. On a 600-point synthetic set the row-order reduction matches a NumPy implementation exactly, and the thread-local reduction stays at an adjusted Rand index of at least 0.98 against that reference.

The timing matrix uses n = 2,600,000 points, k = 24, 30 iterations, and eight standardized features. That k is a compute load. The grouping uses a different feature set, k = 4, and 200,000 trips from 2023. The six grouping features are the zone centroid's easting and northing in the TLC State Plane projection, sine and

cosine of the pickup hour, a weekend indicator, and log(1 + distance). Each column is standardized.

FP-Growth, as a limit

FP-Growth inserts each transaction into a prefix tree and then grows a conditional tree for each frequent item [9]. The conditional trees do not depend on each other. The prefix tree is built in the parent before the pool starts. On the timed input that build is most of the wall time. The input is 1,200,000 transactions at minimum support 0.02. On 1,500 random transactions the miner matches mlxtend, and the parallel miner matches the serial miner. We report the wall times and do not report rules.

Host and protocol

The host reports an Intel Xeon at 2400 MHz, 4 logical CPUs, 4 physical CPUs, and 15.6 GiB of RAM. The model string is the hypervisor name, not a retail SKU. We pin the process to the first min(p, 4) CPUs. Numba reads the CPU count when it is imported, so the thread pool is created with 8 threads before the pin. set_num_threads(p) then selects p of those threads. The p = 8 k-means point is eight threads on four pinned cores.

Every timed configuration has one discarded warmup and then repeated measured runs. The main curves use five measured runs. Chunk-size, block-size, single-month, and weak-scaling runs use three. Every measured run is stored, and the figures plot the median. Error bars on the speedup figure span the first and third quartiles of wall time, converted with the same T1. We do not drop the page cache. The warmup is there so that the measured runs share a warm cache.

Results and Analysis

Cleaning

Of 38,310,226 raw rows, 34,893,980 remain (8.92 percent removed). Invalid codes are the largest cut, 1,740,643 rows (4.54 percent of raw). Duration and non-positive distance each remove 1.24 percent. Within-batch duplicates are 2 rows.

Rule	Rows removed	Share of raw
R1 duration outside 1 min–3 h	475,994	1.24%
R2 non-positive distance	475,927	1.24%
R3 zone id missing from the lookup	398,585	1.04%
R4 fare outside $2.50–$500	322,270	0.84%
R5 speed missing or at least 70 mph	2,825	0.01%
R6 duplicate inside a 1,000,000-row batch	2	0.00%
R7 invalid passenger, rate, or payment code	1,740,643	4.54%
Retained	34,893,980	91.08%

Table 2. Exclusive removals, summed over the twelve files of 2023. Percentages use the raw row total.

Distributions

The median trip is 1.80 miles and 12.6 minutes. The 99th percentiles are 20.3 miles and 65.6 minutes. The median fare is 13.50 dollars and the 99th percentile is 74.40 dollars. Distance and duration both have a long right tail. The cluster fit keeps one log-distance feature so that tail can form its own group, and it does not also include duration and fare, which track the same tail.

The fare histogram is not smooth. The 70.00–70.50 dollar bin holds 1,373,876 trips, 3.9 percent of the 34,864,712 trips in the histogram, against 12,223 and 11,391 in the two adjacent half-dollar bins. A single price carries nearly four percent of the year. That price is the JFK flat fare. We read the spike off the histogram; we do not join it to a tariff table we did not load.

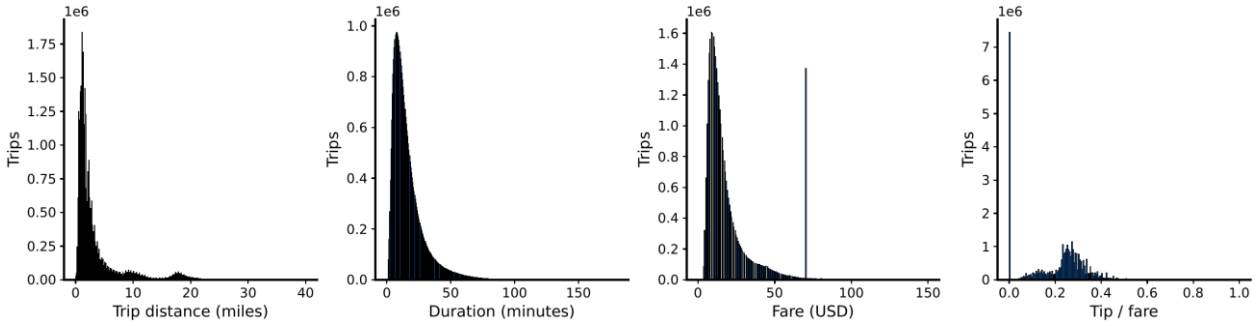


Figure 3. Histograms of the cleaned rows in the twelve 2023 files. Bins are fixed in the ETL.

Time and borough

Of the cleaned pickups with a 2023 timestamp, 34,893,926 falls in the hour-and-borough recount. Manhattan accounts for 31,341,107 of them (89.8 percent). Queens is next, with 3,415,423 (9.79 percent). Brooklyn has 122,590 (0.35 percent), the Bronx 10,401, Staten Island 217, Newark Airport 394, and 3,794 rows have no borough name in the lookup.

The busiest cell is Thursday at 18:00 (405,828 trips). Wednesday at 18:00 is next, with 403,169 trips, 0.7 percent below that cell. Tuesday at 18:00 has 384,280. Weekdays hold 72.9 percent of these pickups. September has 2,551,411 cleaned pickups and May has 3,215,412, which is 26.0 percent higher. The year does not contain a break on the scale of a collapse.

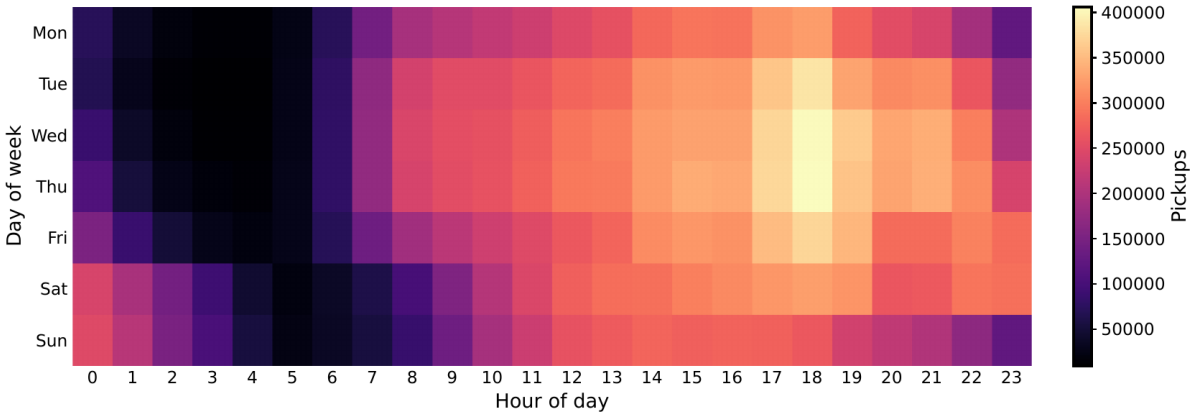


Figure 4. Cleaned 2023 pickups by hour and weekday. The busiest cell is Thursday at 18:00.

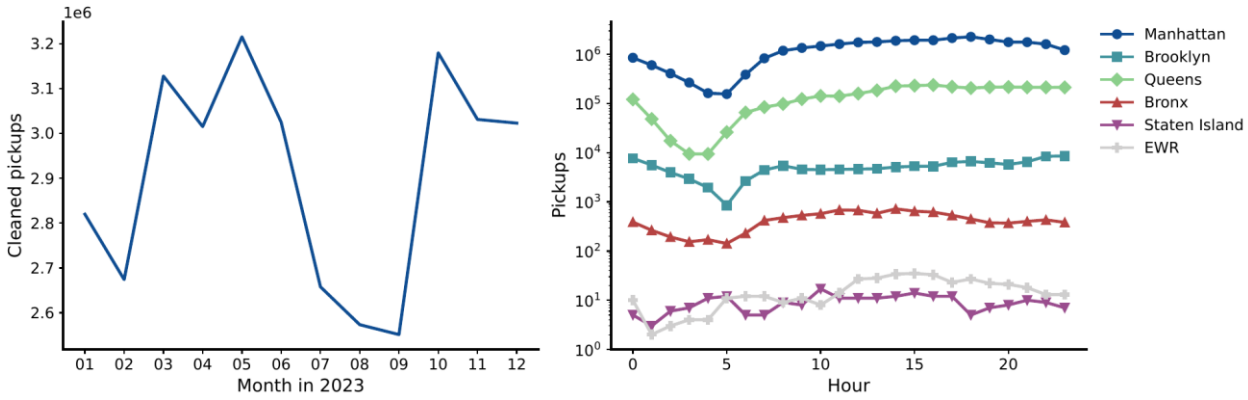


Figure 5. Left: cleaned pickups by month in 2023. Right: the same year by hour and borough, on a logarithmic axis.

Trip groups

The grouping run draws 25,000 cleaned rows from each of the twelve files, keeps the 299,970 rows with a zone centroid, and fits 200,000 of those rows. Silhouette is computed on 8,000 of the fitted rows after 20 iterations. The maximum is k = 2 at 0.550. The score falls to 0.266 at k = 3 and is 0.312 at k = 4 and 0.322 at k = 5.

k	2	3	4	5	6	7	8
Score	0.550	0.266	0.312	0.322	0.308	0.263	0.308

Table 3. Silhouette on 8,000 of the 200,000 fitted rows. The maximum is k = 2. The groups below use k = 4.

The k = 2 fit is a length and borough split. One group has 188,888 rows (94.4 percent), mean distance 2.76 miles, mean hour 14.3, and 95.0 percent Manhattan. The other has 11,112 rows (5.6 percent), mean distance 16.1 miles, mean hour 14.6, and 99.7 percent Queens. The two mean hours differ by 0.3 hours. A higher silhouette here is one airport against the city, not a split by time of day.

We report k = 4, not the silhouette maximum. The k = 5 score is 0.010 higher than k = 4, and the fifth group is a night cluster that mixes weekdays and weekends. Cluster 3 is weekday morning and midday (mean hour 10.5, weekend share 0, midday share 64.7 percent). Cluster 0 is weekday afternoon and evening (mean hour 17.7, weekend share 0, afternoon plus evening 67.8 percent). Cluster 2 is the weekend (weekend share 100 percent, mean hour 13.3). Cluster 1 is the JFK group (99.7 percent Queens, mean distance 16.1 miles, mean duration 42.0 minutes, mean fare 63.69 dollars). Mean distance in the three Manhattan groups stays between 2.68 and 2.84 miles.

	C3 morning	C0 afternoon	C2 weekend	C1 JFK
Rows	58,084	79,950	50,874	11,092
Share	29.0%	40.0%	25.4%	5.5%
Mean distance (miles)	2.81	2.68	2.84	16.10
Mean duration (minutes)	15.8	14.6	13.9	42.0
Mean fare (USD)	17.80	16.67	16.82	63.69
Mean pickup hour	10.5	17.7	13.3	14.6
Weekend share	0%	0%	100%	28.7%
Manhattan share	94.7%	94.9%	95.4%	0%
Queens share	4.9%	4.8%	4.0%	99.7%

Table 4. k-means profiles on 200,000 trips from 2023, k = 4. Column order follows the reading, not the id order.

Cluster 1 is one airport, not the borough. Zone 132, JFK Airport, accounts for 10,903 of its 11,092 rows, or 98.3 percent. The other 189 rows are spread over 41 zones: 161 rows in 24 other Queens zones, and 28 rows in 17 Brooklyn zones. LaGuardia, zone 138, is not in this group. Its 7,243 sampled pickups fall in clusters 0, 2, and 3. Clusters 0, 2, and 3 draw 4.8 percent, 4.0 percent, and 4.9 percent of their rows from Queens, and their mean distance stays between 2.68 and 2.84 miles. Only the JFK trips are long and expensive enough to stand apart. That reading matches the 70-dollar fare spike.

Weekend share is 0 or 1 in the three Manhattan groups because the weekend indicator is one of the six inputs. The morning and afternoon split is not a pre-cut label: the clock enters only as sine and cosine of the hour. The airport group agrees in both geography and distance. k = 2 remains the silhouette maximum, and it is the same airport tail before the clock is split.

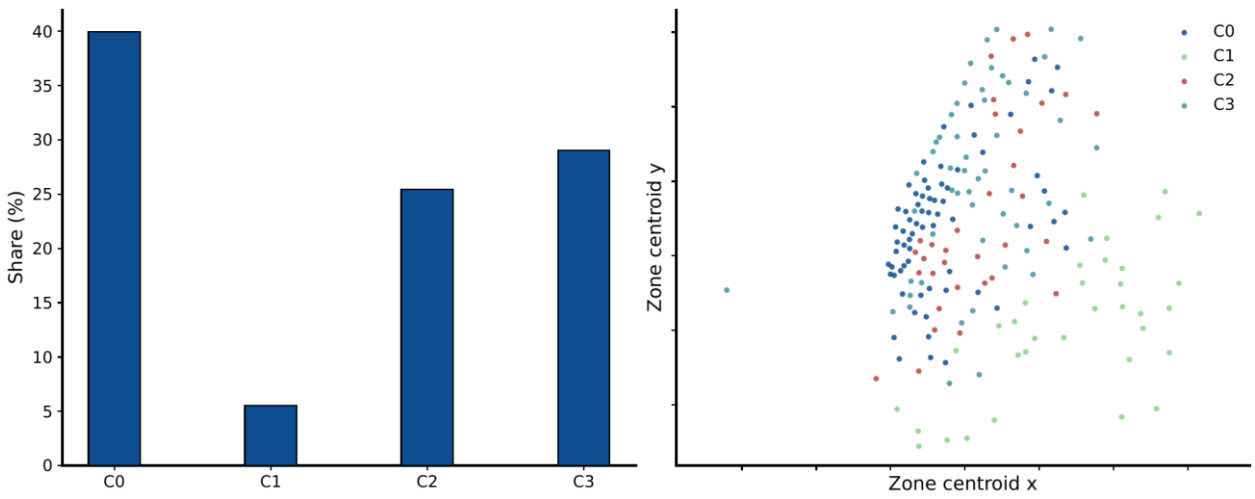


Figure 6. Left: share of the 200,000 fitted rows. Right: zone centroids colored by the majority cluster. Zone 132, JFK Airport, takes cluster 1. A few neighboring Queens zones share that color on a small sample. The weekend group has no zone of its own.

ETL: processes, threads, and chunk size

Figure 7 plots speedup for the ETL, the k-means kernel, and FP-Growth. Every wall time below is a median of the measured repeats. The process pool is close to linear through four workers. Median system CPU moves from 25 percent to 50 percent to 100 percent. At p = 8 the median rises from 13.658 s to 14.266 s and CPU stays at 100 percent. There is no fifth core for the extra processes.

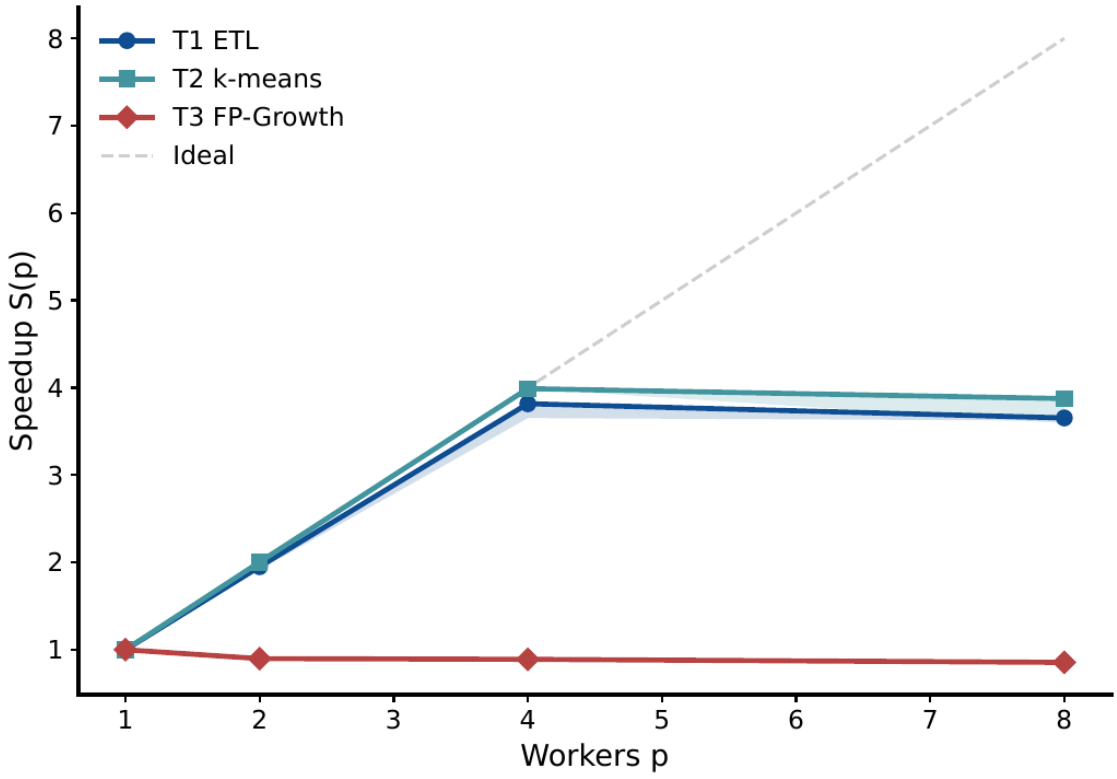


Figure 7. Warm-cache speedup. Bands convert the first and third quartiles of wall time with the same T1. The dashed line is ideal speedup. p = 8 is eight workers on four cores.

p	Median (s)	S(p)	E(p)
1	52.135	1.00	1.000
2	26.717	1.95	0.976
4	13.658	3.82	0.954
8	14.266	3.65	0.457

Table 5. ETL, twelve files of 2023, process pool, chunk size 1. Five repeats.

The same twelve files on a thread pool at p = 4 take 27.423 s, with median CPU 42.3 percent. Relative to the one-process median of 52.135 s, that is S = 1.90. The process pool at the same p takes 13.658 s and uses 100 percent CPU. Pandas and Arrow release the GIL for part of the read and the numeric work, which is why four threads still beat one process. The cleaning rules keep the rest on one interpreter, which is why four processes beat four threads.

A separate pure-Python loop, 400,000 tuples and no NumPy, checks the same mechanism on code that cannot release the GIL. Threads take 0.0179 s at p = 1 and 0.0195 s at p = 4. Processes take 0.292 s and 0.201 s. The pool costs more than the loop, so processes are slower in absolute time. The GIL evidence in Figure 8 is the flat thread bar. The pool decision uses the 27.423 s and 13.658 s above.

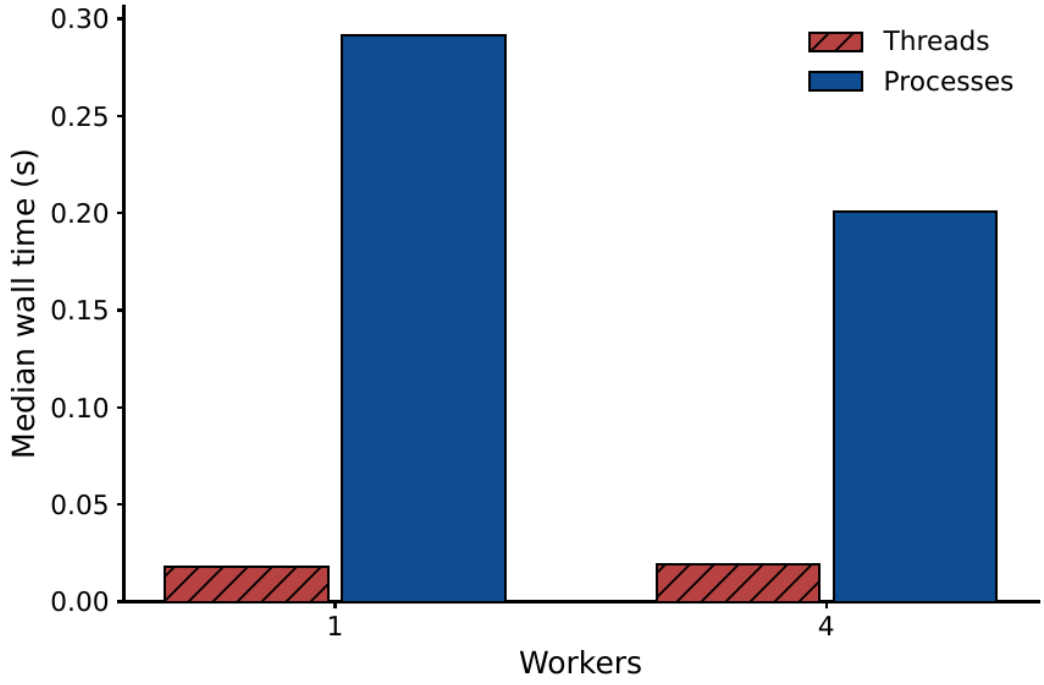


Figure 8. Pure-Python row loop, 400,000 tuples. Threads stay near 0.02 s. Process startup dominates the same loop.

At p = 4, chunk size 1 takes 13.658 s. Chunk size 4 takes 17.038 s, and median CPU falls to 76.2 percent. Chunk size 12 takes 49.668 s, and median CPU is 25.3 percent. Twelve files in one chunk leave a single worker holding the year. A single file, January 2023, takes 4.579 s at p = 1 and 4.195 s at p = 4 (S = 1.09). One file cannot fill four workers, because the file is the unit of work.

Where the ETL time goes

At p = 1 the medians are 1.600 s to read, 19.916 s to clean, and 26.527 s to aggregate. The stack is 48.0 s against a wall of 52.135 s. The gap is pool and interpreter time outside those three timers. Read is a small slice once the cache is warm. At p = 4 the summed clean and aggregate times stay near the serial sums (21.395 s and 27.954 s) while the wall falls to 13.658 s. At p = 8 those sums rise to 42.167 s and 46.218 s, and the wall rises to 14.266 s. The bytes that cross the process boundary are the zone aggregates, not the trips.

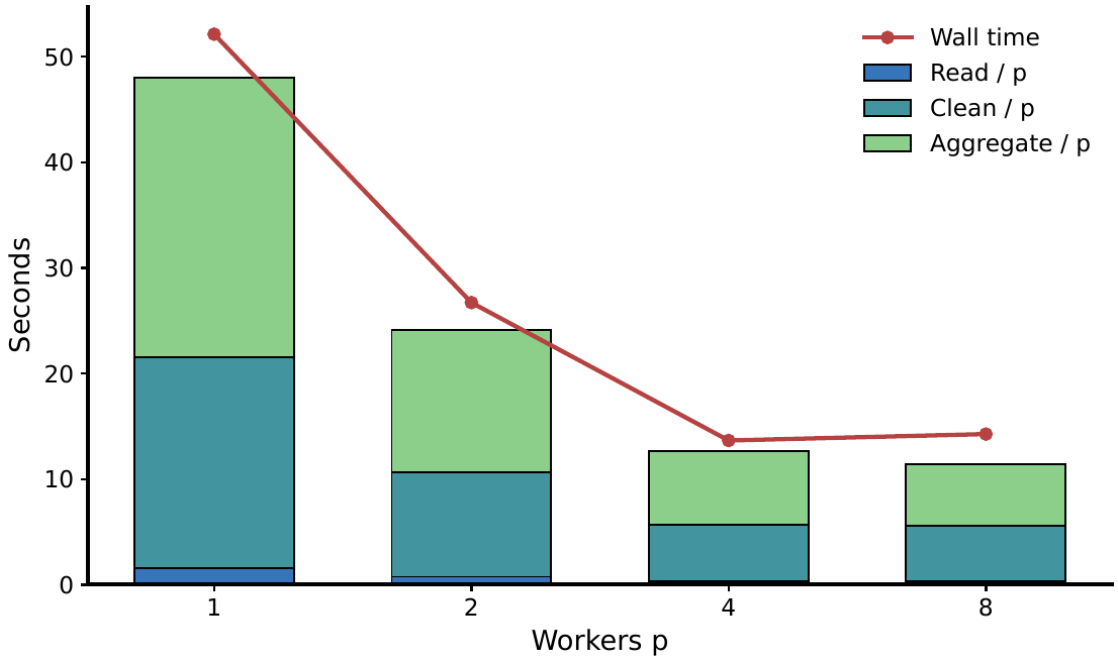


Figure 9. ETL phases for the twelve files of 2023. Each bar is the summed phase time divided by p. The line is wall time.

k-means speedup

The timing kernel is 2,600,000 points, k = 24, 30 iterations, block 8,192, and a thread-local centroid update. This k and these eight features are a compute load. They are not the k = 4 fit.

p	Median (s)	S(p)	E(p)
1	6.428	1.00	1.000
2	3.208	2.00	1.002
4	1.611	3.99	0.997
8	1.659	3.88	0.484

Table 6. k-means, 2,600,000 points, k = 24, 30 iterations, block 8,192. Five repeats.

S(2) = 2.00 and S(4) = 3.99. The p = 2 point is slightly above 2. Replacing each median by the opposite quartile puts that speedup between 1.96 and 2.03, so the excess sits inside the repeat spread. At p = 8 the median rises from 1.611 s to 1.659 s.

Weak scaling keeps about 650,000 points per thread. The medians are 1.407 s at p = 1 (650,000 points), 1.575 s at p = 2 (1,300,000 points), and 1.624 s at p = 4 (2,600,000 points). Time rises 12 percent and then 15 percent while the row count quadruples. Ideal weak scaling would hold the time fixed.

Block size at p = 4, on the same 2,600,000 points, moves the median only from 1.594 s at 64 rows per block to 1.912 s at 262,144 rows. Between 1,024 and 65,536 rows the median stays inside 1.620–1.661 s, next to the 8,192-row point used above (1.611 s). Worker count moves the median from 6.428 s to 1.611 s. Block size does not.

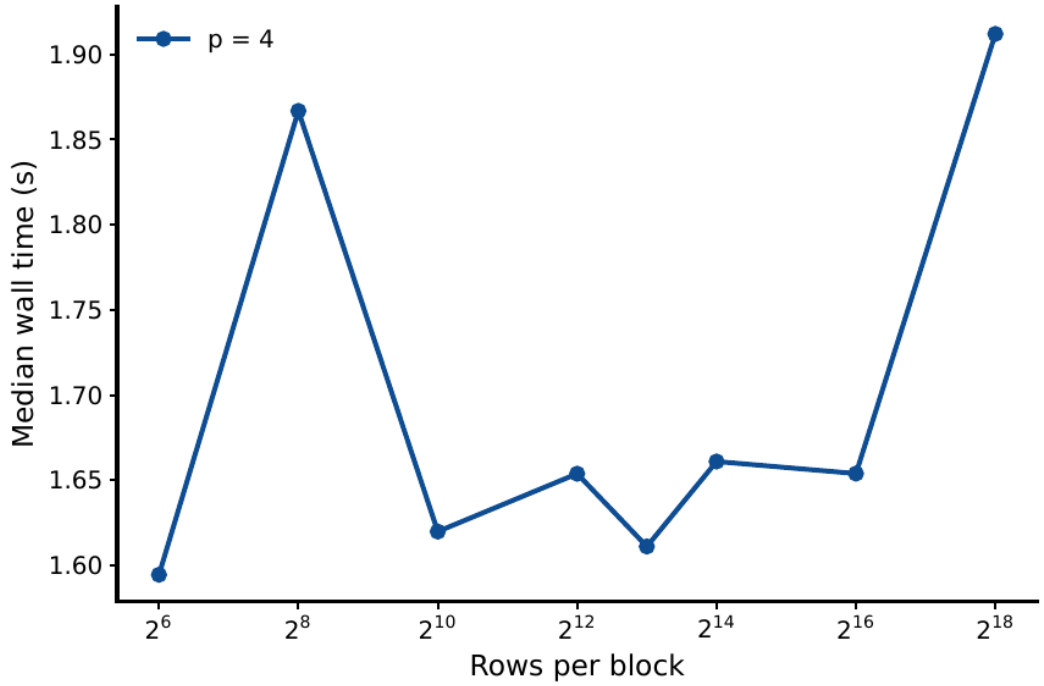


Figure 10. k-means wall time against rows per block, p = 4, three repeats. The vertical range is 1.59–1.91 s.

A split that does not speed up

FP-Growth was timed because the assignment lists association rules. We do not use the itemsets as a finding. The tree is built in the parent before the process pool starts [9]. The pool then grows one conditional tree per frequent item. On 1,200,000 transactions at minimum support 0.02, the parent build dominates the measured

time. Wall time rises from 1.497 s at one worker to 1.666 s, 1.681 s, and 1.750 s. S(4) = 0.89 and S(8) = 0.86. Median system CPU stays between 25.3 percent and 25.6 percent at every p.

p	Median (s)	S(p)	E(p)
1	1.497	1.00	1.000
2	1.666	0.90	0.449
4	1.681	0.89	0.223
8	1.750	0.86	0.107

Table 7. FP-Growth, 1,200,000 transactions, minimum support 0.02. Five repeats.

Discussion

The speedup curves do not support one speedup for the project. At p = 4 the ETL reaches 3.82 and k-means reaches 3.99. Both already sit on a fully busy machine at p = 4. Moving to p = 8 makes each of them slower: 13.658 s to 14.266 s for the ETL, and 1.611 s to 1.659 s for k-means. FP-Growth goes the other way. S(4) = 0.89, and system CPU stays near 25 percent. The code splits into a serial tree build and a process pool, and on this input the pool does not fill the host.

At p = 4 the thread pool takes 27.423 s and the process pool takes 13.658 s. Threads still beat one process (52.135 s), so part of the ETL releases the GIL. The pure-Python loop does not. Chunk size 12 at p = 4 takes 49.668 s with CPU at 25.3 percent, against 13.658 s at chunk size 1. The serial part, in that setting, is the decision to hand every file to one worker.

We did not run Spark. The twelve files finish in under a minute once four processes are used, and the aggregates fit in memory [10]. A larger table would raise that question again. It is unmeasured here.

The study has several limits. Timings share a warm page cache, and the CPU is a virtual Xeon with four cores. k-means assumes spherical clusters in the standardised features. Silhouette uses 8,000 of the 200,000 fitted rows, and the reported labels are a 30-iteration refit rather than the 20-iteration pass that produced the silhouette. Weekend is an input, so a weekend share of 0 or 1 is that coordinate. The zone centroid is the mean of the shapefile vertices, not a count of pickups inside the polygon. Yellow trips are not the full for-hire market. Duplicate removal does not cross the 1,000,000-row batch boundary. In 2023 that boundary hides 2 within-batch duplicates, so a global pass would not move Table 2.

Conclusion

In the twelve yellow-taxi files of 2023, 34,893,980 of 38,310,226 rows survive the seven cleaning rules. Of the dated pickups, 89.8 percent start in Manhattan and 9.79 percent start in Queens. Weekdays hold 72.9

percent, and the busiest cell is Thursday at 18:00 (405,828 trips). May is 26.0 percent above September. There is no within-year break in the monthly series.

k-means on zone location, time of day, weekend, and log distance has its silhouette maximum at k = 2 (0.550). That cut is the JFK tail against the rest, and the two mean hours differ by 0.3 hours. We report k = 4 (silhouette 0.312). The four groups are weekday morning, weekday afternoon and evening, weekend, and a JFK group at 16.1 miles and 63.69 dollars. The weekend group uses an input indicator. The morning and afternoon groups are a split of the hour. The JFK group is the same tail that k = 2 isolates.

On this 4-core host the process-pool ETL moves from 52.135 s at p = 1 to 13.658 s at p = 4 (S = 3.82) and slows to 14.266 s at p = 8. A thread pool at p = 4 takes 27.423 s. Chunk size 12 at p = 4 takes 49.668 s because one worker receives every file. The k-means kernel moves from 6.428 s to 1.611 s (S = 3.99) and slows at p = 8. FP-Growth does not speed up at any p > 1 (S(4) = 0.89), and we do not report its itemsets. The practical rule from these measurements is to put the Python cleaning in a process pool, to keep the chunk size at one file, and to treat a serial tree build as work that more processes will not shrink.

Contribution

All four members contributed equally (25 percent each).

WANG Shen (60216681): 25 percent. Data acquisition, the seven cleaning rules, the process-pool ETL, and integration of the full manuscript.

LIU Wanyang (60915652): 25 percent. Serial and prange k-means, the FP-Growth timing kernel, and the equivalence tests.

WANG Yuwei (60875477): 25 percent. Exploratory figures and the cluster profiles.

YU Xiyan (61076721): 25 percent. The scaling matrix, the speedup analysis, and the slides.

References

[1] New York City Taxi and Limousine Commission, “TLC trip record data,” https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page, 2024, accessed 9 November 2026.

[2] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI), 2004, pp. 137–150.

[3] W. McKinney, “Data structures for statistical computing in Python,” in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.

[4] C. R. Harris, K. J. Millman, S. J. van der Walt et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[5] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in Proceedings of the AFIPS Spring Joint Computer Conference, 1967, pp. 483–485.

[6] S. P. Lloyd, “Least squares quantization in PCM,” IEEE Transactions on Information Theory, vol. 28, no. 2, pp. 129–137, 1982.

[7] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2007, pp. 1027–1035.

[8] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 2015, pp. 1–6.

[9] J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation,” in Proceedings of the ACM SIGMOD International Conference on Management of Data, 2000, pp. 1–12.

[10] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2012, pp. 15–28.